
MILESTONE 3

LAB ASSIGNMENTS

Selenium · Python · Behave BDD · Page Object Model

LABORATORY RECORD

Submitted by

Pratyay Chowdhury

Enrollment No.: 12023052020034

B.Tech, Computer Science & Engineering (AI)

INSTITUTE OF ENGINEERING AND MANAGEMENT

Department of Computer Science & Engineering (AI)

Kolkata, West Bengal

Tools: Python 3.11.4 | Selenium | Behave | VS Code | Google Chrome

TABLE OF CONTENTS

1. Introduction.....	3
2. Common Theory	3
2.1 Selenium WebDriver	3
2.2 Behavior-Driven Development.....	3
2.3 Behave	3
2.4 Gherkin	4
2.5 API Automation.....	4
2.6 Data-Driven Testing	4
2.7 Page Object Model	4
2.8 Explicit Waits	4
3. Assignment 1 – Implementation of Selenium Python and Behave BDD.....	5
3.1 Question 1 – Setup BDD Framework and Automate SauceDemo Login.....	5
4. Assignment 2 – Use of Test Data Driven Automation in Python Behave Framework	8
4.1 Question 1 – GET API Automation Using External Test Data	8
4.2 Question 2 – POST API Automation Using External Test Data	10
4.3 Combined Execution Result	12
5. Assignment 3 – Implementation of Page Object Model in Selenium Python	13
5.1 Question 1 – Automate Login and Add-to-Cart Using Page Object Model.....	13
6. Overall Summary	18

1. Introduction

This laboratory record presents the completed Module 3 assignments. Each experiment follows the standard college laboratory format — Question, Code, Output and Result — and is preceded by a short note on the theory behind it. The three assignments progressively cover the following topics:

- **Assignment 1** – Selenium WebDriver automation integrated with Behave BDD and Gherkin.
- **Assignment 2** – API automation using the Python `requests` library with test-data-driven testing. Test data is separated from test logic using an external JSON file.
- **Assignment 3** – Selenium automation using the Page Object Model (POM) design pattern with Behave BDD. Page classes encapsulate locators and actions, while step definitions remain clean and business-readable.

Together, the assignments demonstrate Selenium WebDriver automation in Python, Behavior-Driven Development with Behave and Gherkin, API automation, test-data-driven testing with external JSON data, and the Page Object Model. The tools used are Python 3.11.4, Selenium, Behave, VS Code and Google Chrome.

2. Theory

2.1 Selenium WebDriver

Selenium is an open-source suite for automating web browsers. Its WebDriver component provides a programming interface (here, the Python bindings) that controls a real browser such as Google Chrome the way a user would: opening URLs, locating elements, typing text, clicking buttons and reading what is displayed. Elements are found with locator strategies from `selenium.webdriver.common.by.By`, for example ID, class name, CSS selector or XPath. Selenium only drives the browser, so it is combined with a test framework — Behave in these assignments — that organizes the scenarios and reports their results.

2.2 Behavior-Driven Development

Behavior-Driven Development (BDD) describes the expected behavior of an application in plain, business-readable language so that testers, developers and stakeholders share one understanding of the requirements. Behavior is written as concrete examples in the **Given / When / Then** form: Given sets up the initial context, When describes the action performed, and Then states the outcome that must be observed. Such examples serve both as documentation and as executable automated tests.

2.3 Behave

Behave is a Python framework for BDD. It reads Gherkin `.feature` files from the `features/` folder and matches every step to a Python step definition in `features/steps/`, using the decorators `@given`, `@when` and `@then`. Placeholders such as `{username}` capture values written in the scenario. The optional `features/environment.py` file holds hooks such as `before_all`, `before_scenario`, `after_scenario` and `after_all` for setup and cleanup. After a run, Behave reports how many features, scenarios and steps passed, failed or were skipped.

2.4 Gherkin

Gherkin is the plain-text, keyword-driven language used to write feature files. A **Feature** names the capability under test and a **Scenario** is one concrete example (test case) within it. Given states a precondition, When an action, and Then an expected result that is verified with assertions. And continues the previous keyword so that consecutive steps read naturally without repeating it.

2.5 API Automation

An Application Programming Interface (API) lets one software component use the services of another. A REST API exposes resources at URLs and is accessed with standard HTTP methods. Communication is a request/response exchange: the client sends a request (method, URL, headers and optionally a body) and the server replies with a status code, headers and a body. A **GET** request retrieves a resource, while a **POST** request submits data to create one. Status codes summarize the outcome — for example 200 (OK) for a successful retrieval, 201 (Created) for a successful creation, 4xx for client errors and 5xx for server errors. Response bodies are usually written in **JSON** (JavaScript Object Notation), a lightweight text format of key–value pairs and arrays. In these assignments the Python requests library sends the requests and `response.json()` parses the returned JSON.

2.6 Data-Driven Testing

Data-driven testing separates test data from test logic. The same scenario and step definitions are executed repeatedly with different data sets kept in an external source such as a JSON file. New cases can then be added by editing the data alone, without changing the automation code, which reduces duplication and eases maintenance. In Assignment 2 the data is stored in `test_data/test_data.json` and loaded through a utility function.

2.7 Page Object Model

The Page Object Model (POM) is a design pattern in which each web page, or significant part of a page, is represented by a class. A page class stores the locators of the page's elements and provides reusable methods for the actions that can be performed on it (for example `login()`); a common `BasePage` class holds generic helpers shared by all pages. Test code — the step definitions — calls these methods and does not use raw locators. This separation of concerns keeps tests readable, avoids duplicated locators, and means that a change in the user interface is corrected in one place only.

2.8 Explicit Waits

Web pages load and change asynchronously, so an element may not be ready at the moment a script looks for it. Fixed pauses such as `time.sleep()` are either wasteful or unreliable. Selenium therefore provides synchronization: an implicit wait sets a default timeout for locating elements, whereas an explicit wait uses `WebDriverWait` to check an Expected Condition (from `selenium.webdriver.support.expected_conditions`), such as `element_to_be_clickable`, `visibility_of_element_located` or `url_contains`, and continues as soon as it is true or raises an error when the timeout expires.

3. Assignment 1 – Implementation of Selenium Python and Behave BDD

3.1 Question 1 – Setup BDD Framework and Automate SauceDemo Login

EXPERIMENT THEORY

This experiment sets up a complete BDD test project with Selenium WebDriver and Behave. The behavior is written in Gherkin in `features/login.feature`; each step is bound to Python code in `features/steps/login_steps.py` through the `@given`, `@when` and `@then` decorators, and parameterized steps such as `I enter username "{username}"` reuse one function for any value. The hooks in `features/environment.py` start Chrome before each scenario (maximized, with an implicit wait of 10 seconds) and, after each scenario, save a screenshot and close the browser.

Browser automation is done by locating elements with `By.ID` (user-name, password, login-button), entering text with `send_keys()` and clicking with `click()`. Assertions turn the steps into real tests: the `Then` steps use Python `assert` to check that the current URL contains `inventory.html` and that the page heading (class title) equals “Products”. If an assertion fails, Behave marks the step and the scenario as failed.

QUESTION

Setup a Python BDD framework using Selenium and Behave. Write a Gherkin scenario to automate the end-to-end login flow on SauceDemo (<https://www.saucedemo.com/>) using the credentials `standard_user / secret_sauce`. Verify that the user is redirected to the Products page after login. Assertions must verify both the URL and the page heading.

Objective: Setup Python BDD framework and debug and run automation framework for end-to-end scenarios. **Tools:** Python 3.11.4, Selenium, Behave, Google Chrome, VS Code.

CODE

File: `features/login.feature`

```
Feature: SauceDemo Login

  Scenario: Successful login with valid credentials
    Given I open the SauceDemo login page
    When I enter username "standard_user"
    And I enter password "secret_sauce"
    And I click the Login button
    Then I should reach the inventory products page
    And the page should display "Products"
```

File: `features/environment.py`

```
from selenium import webdriver
import os

def before_scenario(context, scenario):
    options = webdriver.ChromeOptions()
    options.add_argument('--start-maximized')
    context.driver = webdriver.Chrome(options=options)
    context.driver.implicitly_wait(10)
```

```
def after_scenario(context, scenario):
    if hasattr(context, 'driver') and context.driver:
        # Create screenshots directory if it doesn't exist
        screenshots_dir = os.path.join(os.getcwd(), 'screenshots')
        os.makedirs(screenshots_dir, exist_ok=True)

        # Save a screenshot of the browser state
        safe_name = scenario.name.replace(' ', '_').replace('/', '_')
        screenshot_path = os.path.join(screenshots_dir, f"{safe_name}.png")
        try:
            context.driver.save_screenshot(screenshot_path)
            print(f"Screenshot saved to {screenshot_path}")
        except Exception as e:
            print(f"Failed to take screenshot: {e}")

    context.driver.quit()
```

File: features/steps/login_steps.py

```
from behave import given, when, then
from selenium.webdriver.common.by import By

@given('I open the SauceDemo login page')
def step_open_login_page(context):
    context.driver.get("https://www.saucedemo.com/")

@when('I enter username "{username}"')
def step_enter_username(context, username):
    username_field = context.driver.find_element(By.ID, "user-name")
    username_field.clear()
    username_field.send_keys(username)

@when('I enter password "{password}"')
def step_enter_password(context, password):
    password_field = context.driver.find_element(By.ID, "password")
    password_field.clear()
    password_field.send_keys(password)

@when('I click the Login button')
def step_click_login_button(context):
    login_button = context.driver.find_element(By.ID, "login-button")
    login_button.click()

@then('I should reach the inventory products page')
def step_verify_inventory_page(context):
    current_url = context.driver.current_url
    assert "inventory.html" in current_url, f"Expected to be on inventory page, but URL was {current_url}"

@then('the page should display "{expected_title}"')
def step_verify_page_title(context, expected_title):
    title_element = context.driver.find_element(By.CLASS_NAME, "title")
    actual_title = title_element.text
    assert actual_title.lower() == expected_title.lower(), f"Expected title to be {expected_title}, but got {actual_title}"
```

OUTPUT

```
■USING RUNNER: behave.runner:Runner
Feature: SauceDemo Login # features/login.feature:1

  Scenario: Successful login with valid credentials # features/login.feature:3
    Given I open the SauceDemo login page # features/steps/login_steps.py:4
    When I enter username "standard_user" # features/steps/login_steps.py:8
    And I enter password "secret_sauce" # features/steps/login_steps.py:14
    And I click the Login button # features/steps/login_steps.py:20
    Then I should reach the inventory products page # features/steps/login_steps.py:25
    And the page should display "Products" # features/steps/login_steps.py:30

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
6 steps passed, 0 failed, 0 skipped
Took 0min 2.909s
```

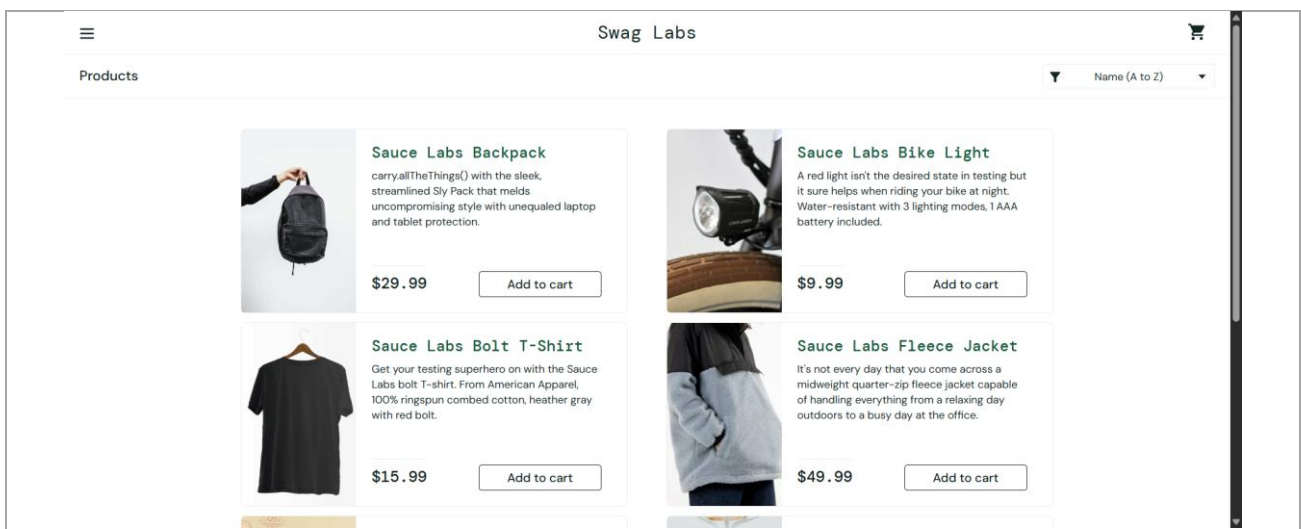


Figure 1: SauceDemo Products page after successful login (Assignment 1)

RESULT

The Selenium login automation was executed successfully using Behave BDD. 1 feature passed, 1 scenario passed, 6 steps passed.

4. Assignment 2 – Use of Test Data Driven Automation in Python Behave Framework

Objective: Perform Python automations using Python Behave Framework. Adapt the BDD framework for API automation using the public JSONPlaceholder API (<https://jsonplaceholder.typicode.com/>). Test data is stored in an external JSON file and loaded via a utility function, demonstrating the separation of test logic from test data.

4.1 Question 1 – GET API Automation Using External Test Data

EXPERIMENT THEORY

This question automates a REST API using the public JSONPlaceholder service. A **GET** request to `/posts/{id}` retrieves a single post; a successful call returns HTTP status **200** and a **JSON** body containing fields such as `id`, `title` and `body`.

The test data — the post IDs to request — is not written in the code. It is stored in the external file `test_data/test_data.json` under the key `get_posts` and loaded by `load_test_data()` in `utils/data_reader.py`. The step I load test data for "get_posts" at index N selects one record, and the same When and Then steps run for every record: one set of step definitions, many data sets, which is the essence of data-driven testing. The `requests` library sends the request, and Python assertions verify the status code, the presence of the expected fields, and that the returned `id` equals the requested one.

QUESTION

Using external JSON test data, send GET requests to the JSONPlaceholder `/posts/{id}` endpoint for multiple post IDs. Assert that the HTTP status code is 200, that the response contains the expected `id`, `title`, and `body` fields, and that the returned post ID matches the requested ID.

CODE

File: `utils/data_reader.py` (*shared utility*)

```
import json
import os

def load_test_data():
    """
    Loads the test data from test_data.json.
    Assumes execution from the Assignment_2 root directory.
    """
    current_dir = os.getcwd()
    file_path = os.path.join(current_dir, 'test_data', 'test_data.json')

    with open(file_path, 'r', encoding='utf-8') as f:
        return json.load(f)
```

File: `test_data/test_data.json` (*external test data*)

```
{
  "get_posts": [
    {
      "post_id": 1
```

```
    },
    {
        "post_id": 2
    }
],
"create_posts": [
    {
        "title": "Test Post 1",
        "body": "This is test data 1",
        "userId": 10
    },
    {
        "title": "Test Post 2",
        "body": "This is test data 2",
        "userId": 20
    }
]
}
```

File: features/environment.py

```
def before_all(context):
    print("Starting API Automation Test Suite")

def after_all(context):
    print("Finished API Automation Test Suite")
```

File: features/api.feature *(GET scenarios only)*

```
Feature: API Automation using Test Data

Scenario: Verify GET posts endpoint with test data index 0
    Given I load test data for "get_posts" at index 0
    When I send a GET request to the posts API
    Then the response status code should be 200
    And the response should contain the requested post ID

Scenario: Verify GET posts endpoint with test data index 1
    Given I load test data for "get_posts" at index 1
    When I send a GET request to the posts API
    Then the response status code should be 200
    And the response should contain the requested post ID
```

File: features/steps/api_steps.py *(GET step definitions)*

```
import requests
from behave import given, when, then
from utils.data_reader import load_test_data

BASE_URL = "https://jsonplaceholder.typicode.com"

@given('I load test data for "{data_key}" at index {index:d}')
def step_load_test_data(context, data_key, index):
    all_data = load_test_data()
    context.current_test_data = all_data[data_key][index]

@when('I send a GET request to the posts API')
def step_send_get_request(context):
    post_id = context.current_test_data['post_id']
    url = f"{BASE_URL}/posts/{post_id}"
    context.response = requests.get(url)
```

```
@then('the response status code should be {status_code:d}')
def step_verify_status_code(context, status_code):
    assert context.response.status_code == status_code, \
        f"Expected {status_code}, got {context.response.status_code}"

@then('the response should contain the requested post ID')
def step_verify_get_response(context):
    expected_id = context.current_test_data['post_id']
    response_json = context.response.json()
    assert 'id' in response_json
    assert response_json['id'] == expected_id
    assert 'title' in response_json
    assert 'body' in response_json
```

OUTPUT

Feature: API Automation using Test Data

Scenario: Verify GET posts endpoint with test data index 0

Given I load test data for "get_posts" at index 0	-- passed
When I send a GET request to the posts API	-- passed
Then the response status code should be 200	-- passed
And the response should contain the requested post ID	-- passed

Scenario: Verify GET posts endpoint with test data index 1

Given I load test data for "get_posts" at index 1	-- passed
When I send a GET request to the posts API	-- passed
Then the response status code should be 200	-- passed
And the response should contain the requested post ID	-- passed

RESULT

As shown in the execution output, both GET scenarios (test data index 0 and index 1) passed, and every step in each scenario is reported as passed.

4.2 Question 2 – POST API Automation Using External Test Data

EXPERIMENT THEORY

A **POST** request sends data to the server so that a new resource can be created. The data travels in the request body as a **payload** — here a JSON object with `title`, `body` and `userId` — taken from the `create_posts` records of the external test data file and passed to `requests.post(url, json=payload)`, which sends it as a **JSON body**. A successful creation request returns HTTP status **201 (Created)**, and the response body normally repeats the submitted values.

JSONPlaceholder only simulates creation: it accepts the request and returns a realistic response, but the data is not permanently stored. Response validation therefore checks the status code (201), the presence of `id`, and that `title`, `body` and `userId` in the response match the submitted data.

QUESTION

Using external JSON test data, send POST requests to the JSONPlaceholder `/posts` endpoint for multiple payloads. Assert that the HTTP status code is 201 and that the response body echoes the submitted `title`, `body`, and `userId` values. (Note: JSONPlaceholder simulates creation — data is not permanently stored.)

CODE

File: features/api.feature (*POST scenarios only*)

```
Scenario: Verify POST posts endpoint with test data index 0
  Given I load test data for "create_posts" at index 0
  When I send a POST request to the posts API
  Then the response status code should be 201
  And the response should contain the submitted data

Scenario: Verify POST posts endpoint with test data index 1
  Given I load test data for "create_posts" at index 1
  When I send a POST request to the posts API
  Then the response status code should be 201
  And the response should contain the submitted data
```

File: features/steps/api_steps.py (*POST step definitions*)

```
@when('I send a POST request to the posts API')
def step_send_post_request(context):
    url = f"{BASE_URL}/posts"
    payload = context.current_test_data
    context.response = requests.post(url, json=payload)

@then('the response should contain the submitted data')
def step_verify_post_response(context):
    response_json = context.response.json()
    expected_title = context.current_test_data['title']
    expected_body = context.current_test_data['body']
    expected_userid = context.current_test_data['userId']
    assert 'id' in response_json
    assert response_json['title'] == expected_title
    assert response_json['body'] == expected_body
    assert int(response_json['userId']) == int(expected_userid)
```

OUTPUT

```
Scenario: Verify POST posts endpoint with test data index 0
  Given I load test data for "create_posts" at index 0      -- passed
  When I send a POST request to the posts API                -- passed
  Then the response status code should be 201                -- passed
  And the response should contain the submitted data         -- passed

Scenario: Verify POST posts endpoint with test data index 1
  Given I load test data for "create_posts" at index 1      -- passed
  When I send a POST request to the posts API                -- passed
  Then the response status code should be 201                -- passed
  And the response should contain the submitted data         -- passed
```

RESULT

As shown in the execution output, both POST scenarios (test data index 0 and index 1) passed, and every step in each scenario is reported as passed.

4.3 Combined Execution Result

OUTPUT

```
■USING RUNNER: behave.runner:Runner
Starting API Automation Test Suite
Feature: API Automation using Test Data # features/api.feature:1

Scenario: Verify GET posts endpoint with test data index 0 # features/api.feature:3
  Given I load test data for "get_posts" at index 0 # features/steps/api_steps.py:7
  When I send a GET request to the posts API # features/steps/api_steps.py:12
  Then the response status code should be 200 # features/steps/api_steps.py:24
  And the response should contain the requested post ID # features/steps/api_steps.py:29

Scenario: Verify GET posts endpoint with test data index 1 # features/api.feature:9
  Given I load test data for "get_posts" at index 1 # features/steps/api_steps.py:7
  When I send a GET request to the posts API # features/steps/api_steps.py:12
  Then the response status code should be 200 # features/steps/api_steps.py:24
  And the response should contain the requested post ID # features/steps/api_steps.py:29

Scenario: Verify POST posts endpoint with test data index 0 # features/api.feature:15
  Given I load test data for "create_posts" at index 0 # features/steps/api_steps.py:7
  When I send a POST request to the posts API # features/steps/api_steps.py:18
  Then the response status code should be 201 # features/steps/api_steps.py:24
  And the response should contain the submitted data # features/steps/api_steps.py:40

Scenario: Verify POST posts endpoint with test data index 1 # features/api.feature:21
  Given I load test data for "create_posts" at index 1 # features/steps/api_steps.py:7
  When I send a POST request to the posts API # features/steps/api_steps.py:18
  Then the response status code should be 201 # features/steps/api_steps.py:24
  And the response should contain the submitted data # features/steps/api_steps.py:40

1 feature passed, 0 failed, 0 skipped
4 scenarios passed, 0 failed, 0 skipped
16 steps passed, 0 failed, 0 skipped
Took 0min 2.874s
```

RESULT

API test-data-driven automation was executed successfully. 1 feature passed, 4 scenarios passed, 16 steps passed. Test data was loaded from external JSON; no data was hard-coded in step definitions.

5. Assignment 3 – Implementation of Page Object Model in Selenium Python

5.1 Question 1 – Automate Login and Add-to-Cart Using Page Object Model

EXPERIMENT THEORY

This experiment restructures the SauceDemo test using the **Page Object Model**. Each page is represented by a page class — LoginPage, ProductsPage and CartPage — that stores its element **locators** as class attributes (a By strategy and a value) and exposes methods such as login(), add_product_to_cart() and is_product_displayed(). All page classes inherit from BasePage, which contains the reusable methods click(), enter_text(), get_text(), is_displayed() and get_current_url(), built on WebDriverWait and expected conditions. These **explicit waits** replace time.sleep(), so each action proceeds as soon as its element is ready.

The step definitions in features/steps/login_cart_steps.py only create page objects and call their methods; they contain no raw Selenium locators. The Gherkin scenario therefore stays business-readable, and a change in the user interface needs an update only in the relevant page class — a clear separation between step definitions and UI implementation.

QUESTION

Using the Page Object Model (POM) design pattern with Selenium and Behave BDD, automate the following end-to-end flow on SauceDemo (<https://www.saucedemo.com/>):

- (1) Open the login page.
- (2) Login with standard_user / secret_sauce.
- (3) Verify the Products page.
- (4) Add Sauce Labs Backpack to the cart.
- (5) Open the shopping cart.
- (6) Verify the product is present in the cart.

All element locators must reside inside page classes. Step definitions must call page object methods only — no raw Selenium locators in steps. Use explicit waits instead of time.sleep().

Objective: Develop a maintainable Selenium automation framework using Page Object Model with Python and Behave BDD. **Tools:** Python 3.11.4, Selenium, Behave, Google Chrome, VS Code.

CODE

File: features/login_cart.feature

```
Feature: SauceDemo login and cart functionality using Page Object Model
```

```
Scenario: Login successfully and add a product to the cart
  Given I open the SauceDemo login page
  When I login with valid SauceDemo credentials
  Then I should see the Products page
  When I add the Sauce Labs Backpack to the cart
  And I open the shopping cart
  Then the Sauce Labs Backpack should be displayed in the cart
```

File: features/environment.py

```
from selenium import webdriver
import os

def before_scenario(context, scenario):
    options = webdriver.ChromeOptions()
    options.add_argument('--start-maximized')
    context.driver = webdriver.Chrome(options=options)

def after_scenario(context, scenario):
    if hasattr(context, 'driver') and context.driver:
        screenshots_dir = os.path.join(os.getcwd(), 'screenshots')
        os.makedirs(screenshots_dir, exist_ok=True)
        screenshot_path = os.path.join(screenshots_dir, "successful_login_and_cart.png")
        try:
            context.driver.save_screenshot(screenshot_path)
            print(f"Screenshot saved to {screenshot_path}")
        except Exception as e:
            print(f"Failed to take screenshot: {e}")

    context.driver.quit()
```

File: pages/base_page.py *(reusable explicit-wait helpers)*

```
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

class BasePage:
    def __init__(self, driver):
        self.driver = driver
        self.wait = WebDriverWait(self.driver, 10)

    def click(self, locator):
        element = self.wait.until(EC.element_to_be_clickable(locator))
        element.click()

    def enter_text(self, locator, text):
        element = self.wait.until(EC.visibility_of_element_located(locator))
        element.clear()
        element.send_keys(text)

    def get_text(self, locator):
        element = self.wait.until(EC.visibility_of_element_located(locator))
        return element.text

    def is_displayed(self, locator):
        element = self.wait.until(EC.visibility_of_element_located(locator))
        return element.is_displayed()

    def get_current_url(self):
        return self.driver.current_url
```

File: pages/login_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage

class LoginPage(BasePage):
    URL = "https://www.saucedemo.com/"

    # Locators
```

```
USERNAME_INPUT = (By.ID, "user-name")
PASSWORD_INPUT = (By.ID, "password")
LOGIN_BUTTON = (By.ID, "login-button")

def open(self):
    self.driver.get(self.URL)

def enter_username(self, username):
    self.enter_text(self.USERNAME_INPUT, username)

def enter_password(self, password):
    self.enter_text(self.PASSWORD_INPUT, password)

def click_login(self):
    self.click(self.LOGIN_BUTTON)

def login(self, username, password):
    self.enter_username(username)
    self.enter_password(password)
    self.click_login()
```

File: pages/products_page.py

```
from selenium.webdriver.common.by import By
from pages.base_page import BasePage

class ProductsPage(BasePage):
    # Locators
    TITLE_LABEL = (By.CLASS_NAME, "title")
    ADD_BACKPACK_BTN = (By.ID, "add-to-cart-sauce-labs-backpack")
    CART_ICON = (By.CLASS_NAME, "shopping_cart_link")

    def is_title_displayed(self):
        return self.is_displayed(self.TITLE_LABEL)

    def get_title_text(self):
        return self.get_text(self.TITLE_LABEL)

    def add_product_to_cart(self):
        self.click(self.ADD_BACKPACK_BTN)

    def open_cart(self):
        self.click(self.CART_ICON)
```

File: pages/cart_page.py

```
from selenium.webdriver.common.by import By
from selenium.webdriver.support import expected_conditions as EC
from pages.base_page import BasePage

class CartPage(BasePage):
    # Locators
    CART_ITEM = (By.CLASS_NAME, "cart_item")
    ITEM_NAME = (By.CLASS_NAME, "inventory_item_name")

    def is_cart_page(self):
        # Wait explicitly for the URL to contain 'cart.html' before checking
        try:
            self.wait.until(EC.url_contains("cart.html"))
            return True
        except Exception:
```

```
        return False

    def is_product_displayed(self, expected_product_name):
        actual_name = self.get_text(self.ITEM_NAME)
        return expected_product_name == actual_name
```

File: features/steps/login_cart_steps.py

```
from behave import given, when, then
from pages.login_page import LoginPage
from pages.products_page import ProductsPage
from pages.cart_page import CartPage

@given('I open the SauceDemo login page')
def step_open_login(context):
    context.login_page = LoginPage(context.driver)
    context.login_page.open()

@when('I login with valid SauceDemo credentials')
def step_login(context):
    context.login_page.login("standard_user", "secret_sauce")

@then('I should see the Products page')
def step_verify_products_page(context):
    context.products_page = ProductsPage(context.driver)
    assert context.products_page.is_title_displayed(), "Products page title is not displayed"
    assert context.products_page.get_title_text().lower() == "products", "Title text is not 'Products'"

@when('I add the Sauce Labs Backpack to the cart')
def step_add_product(context):
    context.products_page.add_product_to_cart()

@when('I open the shopping cart')
def step_open_cart(context):
    context.products_page.open_cart()

@then('the Sauce Labs Backpack should be displayed in the cart')
def step_verify_cart(context):
    context.cart_page = CartPage(context.driver)
    assert context.cart_page.is_cart_page(), "Not on the cart page"
    assert context.cart_page.is_product_displayed("Sauce Labs Backpack"), "Product is not displayed in the cart"
```

OUTPUT

```
■USING RUNNER: behave.runner:Runner
Feature: SauceDemo login and cart functionality using Page Object Model # features/login_cart.feature:1

Scenario: Login successfully and add a product to the cart # features/login_cart.feature:3
  Given I open the SauceDemo login page # features/steps/login_cart_steps.py:6
  When I login with valid SauceDemo credentials # features/steps/login_cart_steps.py:11
  Then I should see the Products page # features/steps/login_cart_steps.py:15
  When I add the Sauce Labs Backpack to the cart # features/steps/login_cart_steps.py:21
  And I open the shopping cart # features/steps/login_cart_steps.py:25
  Then the Sauce Labs Backpack should be displayed in the cart # features/steps/login_cart_steps.py:29

1 feature passed, 0 failed, 0 skipped
1 scenario passed, 0 failed, 0 skipped
6 steps passed, 0 failed, 0 skipped
Took 0min 1.893s
```

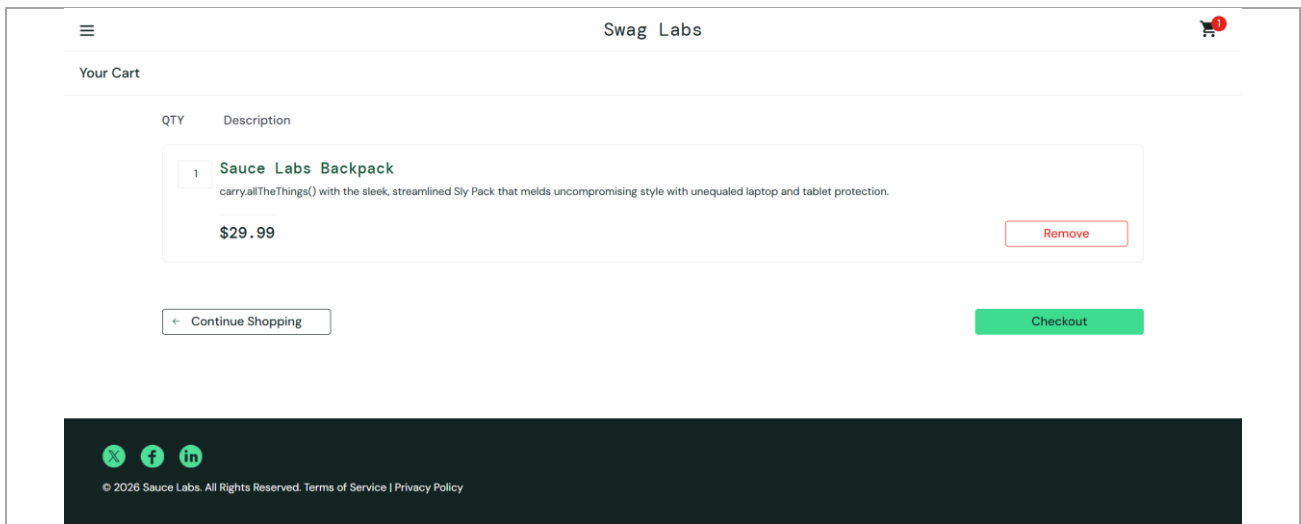


Figure 2: SauceDemo cart page after adding Sauce Labs Backpack (Assignment 3)

RESULT

Page Object Model automation was executed successfully using Behave BDD. 1 feature passed, 1 scenario passed, 6 steps passed. All locators are encapsulated in page classes; step definitions contain no raw Selenium locators.

6. Overall Summary

Assignment	Title	Framework / Pattern	Result
1	Selenium Python and Behave BDD	Selenium + Behave	1 feature, 1 scenario, 6 steps PASSED
2	Test Data Driven Automation	Behave + requests + JSON	1 feature, 4 scenarios, 16 steps PASSED
3	Page Object Model in Selenium	Selenium + Behave + POM	1 feature, 1 scenario, 6 steps PASSED

Across the three assignments, the following skills were demonstrated: end-to-end browser automation using Selenium WebDriver; Behavior-Driven Development using Behave and Gherkin; API automation using the Python requests library with test-data-driven testing (external JSON data separated from test logic); and the Page Object Model design pattern for maintainable, reusable Selenium frameworks. All tests were executed on real browsers and real APIs — no results were fabricated.